操作系统实验 Lab 2 实验报告

Uthread: switching between threads

实现思路

本实验补全用户级线程库，让三个线程使用各自的栈轮流执行。线程调用 `thread_yield()` 后让出 CPU，再次被调度时从上次暂停的位置继续运行。

修改的文件为 `user/uthread.c` 和 `user/uthread_switch.S`。每个线程增加一个上下文结构体，保存 `ra`、`sp` 和 `s0～s11`。`ra` 保存返回地址，`sp` 保存栈指针，`s0～s11` 按调用约定需要由被调用者保持。切换发生在函数调用处，其余 caller-saved 寄存器由调用方按调用约定处理。

关键代码

创建线程时，在 `thread_create()` 中设置初始上下文：

```
memset(&t->context, 0, sizeof(t->context));
t->context.ra = (uint64)func;
t->context.sp = (uint64)(t->stack + STACK_SIZE) & ~((uint64)15);
t->state = RUNNABLE;
```

新线程还没有执行过，将 `ra` 设为入口函数地址后，第一次切换中的 `ret` 就会跳到该函数。栈向低地址增长，因此 `sp` 从栈数组末尾开始，并向下对齐到 16 字节边界。

`thread_schedule()` 找到下一个可运行线程后，更新当前线程指针，再调用切换函数：

```
next_thread->state = RUNNING;
t = current_thread;
current_thread = next_thread;
thread_switch(&t->context, &next_thread->context);
```

汇编函数的 `a0` 指向旧线程上下文，`a1` 指向新线程上下文。先用 `sd` 保存旧线程寄存器，再用 `ld` 恢复新线程寄存器，最后执行 `ret`。以 `ra` 和 `sp` 为例：

```
# 保存阶段
sd ra, 0(a0)
sd sp, 8(a0)

# 恢复阶段
ld ra, 0(a1)
ld sp, 8(a1)
```

完整代码还保存和恢复 `s0～s11`，偏移为 16～104，每个字段占 8 字节，与 C 结构体中的排列一致。恢复 `sp` 后使用的是新线程的调用栈，恢复 `ra` 后，`ret` 返回到新线程上次调用 `thread_switch()` 后的位置，随后逐层返回到线程函数。

测试结果

运行官方测试：

```
make grade GRADEFLAGS='-v uthread'
```

结果为：

```
uthread: OK (2.5s)
```

测试日志中的部分输出如下：

```
$ uthread
thread_a started
thread_b started
thread_c started
thread_c 0
thread_a 0
thread_b 0
thread_c 1
thread_a 1
thread_b 1
...
thread_c 99
thread_a 99
thread_b 99
thread_c: exit after 100
thread_a: exit after 100
thread_b: exit after 100
thread_schedule: no runnable threads
```

三个线程都完成了 100 次循环。最后一个线程退出后，没有可运行线程，程序打印 `no runnable threads` 并结束。